



Lab 1C Errors Encountered with Terraform

This page documents errors encountered during `terraform apply` and their solutions.

Error 1: Subnet CIDR Conflict

The Error

```
Error: creating EC2 Subnet: InvalidSubnet.Conflict: The CIDR
'10.212.2.0/24' conflicts with another subnet
```

What Went Wrong

You're trying to create a subnet with a CIDR block that **already exists** in your VPC. This happens when:

- A previous `terraform apply` partially succeeded and created some subnets
- You have subnets from Lab 1A/1B still in the VPC
- Your `public_subnet_cidrs` or `private_subnet_cidrs` variable has overlapping or duplicate values

The Fix

Option A: Import existing subnets into Terraform state

```
# Find existing subnet IDs
aws ec2 describe-subnets --filters "Name=cidr-block,Values=1
0.212.2.0/24" --query "Subnets[].SubnetId"
```

```
# Import into Terraform
terraform import 'aws_subnet.helga_public_subnets[2]' subnet-
xxxxxxxxxx
```

Option B: Delete the conflicting subnet manually

```
aws ec2 delete-subnet --subnet-id subnet-xxxxxxxxxx
```

Option C: Use different CIDR blocks in your variables.tf

Make sure your subnet CIDRs don't overlap with existing subnets.

I decided to change the subnets to 10.241.

Error 2: AMI/Instance Architecture Mismatch

The Error

```
Error: creating EC2 Instance: InvalidParameterValue: The arch
itecture 'x86_64' of the specified instance type does not mat
ch the architecture 'arm64' of the specified AMI.
```

What Went Wrong

Your AMI is built for **ARM64** (Graviton) processors, but your instance type (likely `t2.micro` or `t3.micro`) uses **x86_64** architecture. These must match.

The Fix

Option A: Change instance type to Graviton (ARM64)

In your `terraform.tfvars` or `variables.tf`:

```
ec2_instance_type = "t4g.micro" # Graviton instance (ARM64)
```

Option B: Change AMI to x86_64

Find an x86_64 AMI for Amazon Linux 2023:

```
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=al2023-ami-*-x86_64" "Name=state,Values=available" \
  --query "sort_by(Images, &CreationDate)[-1].ImageId" \
  --output text
```

Then update your variable:

```
ec2_ami_id = "ami-xxxxxxxx" # Use the x86_64 AMI ID
```

ami-06f1fc9ae5ae7f31e

Quick Reference

Architecture	Instance Types	AMI Type
x86_64	t2., t3., m5., c5.	Standard Intel/AMD AMIs
arm64	t4g., m6g., c6g.*	Graviton AMIs

Error 3: SSM Parameters Already Exist

The Error

```
Error: creating SSM Parameter (/lab/db/endpoint): ParameterAlreadyExists: The parameter already exists.
```

What Went Wrong

These SSM parameters were created in **Lab 1B** and still exist in your AWS account. Terraform doesn't know about them because they weren't created by this Terraform configuration.

The Fix

Option A: Delete existing parameters (Recommended for clean slate)

```
aws ssm delete-parameter --name "/lab/db/endpoint"  
aws ssm delete-parameter --name "/lab/db/port"  
aws ssm delete-parameter --name "/lab/db/name"
```

Option B: Import into Terraform state

```
terraform import aws_ssm_parameter.helga_db_endpoint_param "/lab/db/endpoint"  
terraform import aws_ssm_parameter.helga_db_port_param "/lab/db/port"  
terraform import aws_ssm_parameter.helga_db_name_param "/lab/db/name"
```

Option C: Add overwrite = true to your Terraform

```
resource "aws_ssm_parameter" "helga_db_endpoint_param" {  
  name      = "/lab/db/endpoint"  
  type      = "String"  
  value     = aws_db_instance.helga_rds01.address  
  overwrite = true # <-- ADD THIS LINE  
  ...  
}
```

After running both commands just to see what it would say, it confirms with me that it does not exist. I definitely added `overwrite = true` just as an additional to make sure that I wouldn't encounter this error again.

Recommended Recovery Sequence

1. **Fix the AMI/Instance mismatch first** (easiest, just change a variable)
2. **Delete or import SSM parameters**
3. **Resolve subnet conflict** (delete old subnet or use different CIDRs)

4. Run `terraform plan` to verify no more errors
 5. Run `terraform apply` again
-

Lesson Learned



Lab 1A/1B resources conflict with Lab 1C Terraform

When transitioning from console-created resources (Lab 1A/1B) to Infrastructure as Code (Lab 1C), you must either:

- **Destroy** the old resources before Terraform creates new ones
- **Import** existing resources into Terraform state
- **Use different names/CIDRs** to avoid conflicts

This is a common real-world scenario when "lifting" existing infrastructure into IaC.

Error 4: ENI Detach AuthFailure (RDS-Managed ENI)

The Error

```
Error: deleting ENIs using Security Group (sg-086704e9d1d64764c): 1 error occurred:
```

```
* detaching RDS ENI (eni-0145de91fd6c72657): api error AuthFailure: You do not have permission to access the specified resource.
```

What Went Wrong

Terraform is trying to **detach an Elastic Network Interface (ENI)** that belongs to RDS. The problem is:

- **RDS-managed ENIs cannot be manually detached** — AWS manages them automatically

- When you delete an RDS instance, AWS handles the ENI cleanup
- This error typically occurs when Terraform tries to delete a Security Group or Subnet that still has an RDS ENI attached

Why This Happens

Terraform's dependency graph sometimes tries to delete resources in the wrong order:

1. Terraform tries to delete the Security Group
2. But RDS still has an ENI using that Security Group
3. Terraform tries to force-detach the ENI
4. AWS blocks it because RDS owns that ENI

The Fix

Option A: Delete the RDS instance first (Recommended)

RDS must be deleted before its dependent resources (SG, Subnet). Either:

1. Run `terraform destroy` and let it complete (may take 5-10 min for RDS)
2. Or manually delete RDS first:

```
# Delete RDS instance (takes several minutes)
aws rds delete-db-instance \
  --db-instance-identifier lab1c-rds01 \
  --skip-final-snapshot \
  --delete-automated-backups

# Wait for deletion to complete
aws rds wait db-instance-deleted --db-instance-identifier lab
1c-rds01
```

Option B: Use `terraform destroy -target` to control order

```
# Destroy RDS first
terraform destroy -target=aws_db_instance.helga_rds01
```

```
# Then destroy everything else
terraform destroy
```

Option C: Wait and retry

If RDS is already being deleted, just wait 5-10 minutes and run `terraform apply` or `terraform destroy` again. Once RDS is gone, the ENI will be released.

Key Insight



You cannot manually detach RDS ENIs.

AWS manages them. The only way to release them is to delete the RDS instance itself.

Error 5: SSM Parameter Still Exists (Persistent)

The Error

```
Error: creating SSM Parameter (/lab/db/name): ParameterAlreadyExists: The parameter already exists.
```

What Went Wrong

Despite adding `overwrite = true` to your Terraform config, this parameter still exists and is blocking creation. Possible causes:

1. **Terraform state mismatch** — Terraform doesn't know the parameter exists
2. **The `overwrite = true` change wasn't applied yet** — You need to run `terraform apply` after making the change
3. **Partial apply** — The parameter was created in a previous run but Terraform lost track of it

The Fix

Option A: Force delete and re-run

```
# Delete the stubborn parameter
aws ssm delete-parameter --name "/lab/db/name"

# Verify it's gone
aws ssm get-parameter --name "/lab/db/name" # Should return
error

# Re-run Terraform
terraform apply
```

Option B: Import into Terraform state

```
terraform import aws_ssm_parameter.helga_db_name_param "/lab/
db/name"
```

Option C: Verify `overwrite = true` is in your code

Make sure your Terraform has this:

```
resource "aws_ssm_parameter" "helga_db_name_param" {
  name      = "/lab/db/name"
  type      = "String"
  value     = var.db_name
  overwrite = true # <-- MUST BE HERE
  tags = {
    Name = "${local.name_prefix}-param-db-name"
  }
}
```

Then run `terraform plan` to confirm the change is detected.

Updated Recovery Sequence (Round 2)

1. **Wait for any in-progress RDS deletion** (if applicable)

2. Delete stubborn SSM parameter manually:

```
aws ssm delete-parameter --name "/lab/db/name"
```

3. Run `terraform plan` to see clean state

4. Run `terraform apply` again

Round 3: Progress Check

SSM Parameter - RESOLVED

When running `aws ssm delete-parameter --name "/lab/db/name"`, you got:

```
An error occurred (ParameterNotFound) when calling the Delete
Parameter operation:
```

This is good news! The parameter doesn't exist, so it can be created fresh. The previous Terraform error was likely stale from a prior run.

ENI Detach - STILL BLOCKING

The same ENI error persists because **RDS is still alive** (or mid-deletion). The ENI (`eni-0145de91fd6c72657`) is still attached to RDS.

Next Steps

1. Check if RDS exists:

```
aws rds describe-db-instances --db-instance-identifier lab1c-
rds01 --query "DBInstances[].DBInstanceStatus"
```

2. If RDS is still "available" or "creating", delete it:

```
aws rds delete-db-instance \
  --db-instance-identifier lab1c-rds01 \
```

```
--skip-final-snapshot \  
--delete-automated-backups
```

3. If RDS is "deleting", just wait 5-10 minutes

4. Once RDS is gone, re-run:

```
terraform apply
```



RDS deletion takes 5-10 minutes. Don't keep retrying Terraform until RDS is fully deleted. You can monitor with:

```
aws rds describe-db-instances --db-instance-identifie  
r helga-rds01
```

When it returns "DBInstanceNotFound", you're clear.

Error 6: DB Subnet Group - Subnet Reference Issue

The Error

```
Error with aws_db_subnet_group.helga_rds_subnet_group01
```

What Went Wrong

The `subnet_ids` reference in your DB subnet group is pointing to subnets that either:

1. **Don't exist yet** — The private subnets failed to create in an earlier apply
2. **Have a naming mismatch** — Case sensitivity issue (e.g., `Helga` vs `helga`)
3. **Were deleted** — Previous cleanup removed the subnets but Terraform state still expects them

Check Your Terraform Code

Your DB subnet group should look like this:

```
resource "aws_db_subnet_group" "helga_rds_subnet_group01" {
  name          = "${local.name_prefix}-rds-subnet-group01"
  subnet_ids    = aws_subnet.helga_private_subnets[*].id    # <--
Check this line
  tags = {
    Name = "${local.name_prefix}-rds-subnet-group01"
  }
}
```

Common Issues

Problem	Symptom	Fix
Case mismatch	<code>Helga_private_subnets</code> vs <code>helga_private_subnets</code>	Make all resource names consistent (lowercase recommended)
Missing subnets	Subnets failed to create earlier	Run <code>terraform plan</code> to see if subnets will be created
Splat syntax	<code>[*].id</code> not resolving	Ensure <code>aws_subnet.helga_private_subnets</code> exists and is a list

The Fix

Step 1: Verify your private subnet resource name

In your `04-Main.tf`, find your private subnet resource:

```
resource "aws_subnet" "helga_private_subnets" {    # <-- Note
the exact name
  count          = length(var.private_subnet_cidrs)
  ...
}
```

Step 2: Make sure the DB subnet group references the EXACT same name

```
subnet_ids = aws_subnet.helga_private_subnets[*].id
#           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#           Must match exactly (case-sensitive)
```

Step 3: Check that private subnets will be created

```
terraform plan | grep -A5 "aws_subnet.helga_private_subnets"
```

If no private subnets show up, verify your `private_subnet_cidrs` variable is defined.

Quick Diagnostic

```
# List what Terraform knows about your subnets
terraform state list | grep subnet
```

If the private subnets aren't in state, they need to be created first.

Resolution: Fresh Start Approach

Decision

After multiple partial applies and state issues, the cleanest path forward is:

1. **Tear down everything** via `terraform destroy` (wait for RDS to fully delete)
2. **Clear Terraform state** if needed: `rm -rf .terraform terraform.tfstate*`
3. **Re-initialize:** `terraform init`
4. **Fresh apply:** `terraform apply`

Pre-Flight Checklist Before Fresh Apply

- ☐ VPC CIDR updated (now using `10.241.x.x`)
- ☐ AMI matches instance architecture (x86_64 AMI + t2/t3 instance OR arm64 AMI + t4g instance)
- ☐ `user_data.sh` has correct `DB_SECRET_NAME="helga/rds/mysql"`

- ☐ systemd service also has `Environment="DB_SECRET_NAME=helga/rds/mysql"`
- ☐ No stray spaces in systemd unit file (`After= network.target` not `After= network.target )`
- ☐ CloudWatch namespace in `user_data.sh` matches Terraform alarm (`Lab/RDSApp`)
- ☐ SSM parameters have `overwrite = true`

Why Fresh Start Works



Sometimes the cleanest fix is starting over.

When Terraform state gets out of sync with reality (partial applies, manual deletions, imports), a fresh `destroy` + `apply` cycle eliminates drift and gives you a known-good baseline.

Error 7: Secret Scheduled for Deletion (Invisible but Blocking)

The Error

```
Error: creating Secrets Manager Secret (helga/rds/mysql): InvalidRequestException: You can't create this secret because a secret with this name is already scheduled for deletion.
```

What Went Wrong

When you delete a secret in AWS Secrets Manager, it **doesn't delete immediately**. Instead:

1. The secret enters a "scheduled for deletion" state
2. Default recovery window is **30 days** (minimum 7 days)
3. During this time, the secret is **hidden from the console** but still exists
4. You **cannot create a new secret with the same name** until deletion completes

This is why you don't see it in the console — it's in deletion limbo.

The Fix

Force delete the secret immediately (bypasses the recovery window):

```
aws secretsmanager delete-secret \  
  --secret-id "helga/rds/mysql" \  
  --force-delete-without-recovery
```

Verify It's Gone

```
aws secretsmanager describe-secret --secret-id "helga/rds/mysql"
```

Should return: `ResourceNotFoundException`

Then Re-Run Terraform

```
terraform apply
```



Warning: `--force-delete-without-recovery` is permanent. The secret cannot be recovered after this. Only use when you're sure you don't need the old secret data.

Why This Happens

AWS designed Secrets Manager with a recovery window to prevent accidental data loss. This is a safety feature, but it can block IaC workflows when you're iterating quickly.

Issue 8: App Not Responding (Runtime Debugging)

The Symptom

Terraform applied successfully, but the app doesn't return any data when you hit the endpoints.

Likely Causes

Cause	How to Check
Flask app not running	<code>systemctl status notes-app</code>
Port 5000 not exposed in SG	Check EC2 Security Group inbound rules
RDS not ready yet	RDS takes 5-10 min after creation
Secret has wrong values	<code>aws secretsmanager get-secret-value --secret-id helga/rds/mysql</code>
<code>user_data.sh</code> failed	Check <code>/var/log/user_data.log</code>

⚠️ LIKELY ISSUE: Port Mismatch

Your **Flask app runs on port 5000**, but your Terraform Security Group only allows **port 80**:

```
ingress {
  from_port = 80    # <-- This is wrong
  to_port   = 80
  ...
}
```

Fix: Add port 5000 to your EC2 Security Group:

```
ingress {
  description = "Flask app"
  from_port   = 5000
  to_port     = 5000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
```

Or change via AWS CLI:

```
aws ec2 authorize-security-group-ingress \
  --group-id <your-ec2-sg-id> \
  --protocol tcp \
  --port 5000 \
  --cidr 0.0.0.0/0
```

SSH Troubleshooting Commands

SSH into your EC2 and run:

```
# 1. Check if user_data completed
cat /var/log/user_data.log

# 2. Check if Flask service is running
systemctl status notes-app

# 3. Check Flask app logs
journalctl -u notes-app -f

# 4. Test locally (bypasses Security Group)
curl http://localhost:5000/health

# 5. Test DB connection
curl -X POST http://localhost:5000/init
```

Expected Results

Command	Expected Output
<code>curl localhost:5000/health</code>	<code>{"status": "healthy", "service": "notes-app"}</code>
<code>curl -X POST localhost:5000/init</code>	<code>{"status": "success", "message": "Database initialized"}</code>

If `/health` works but `/init` fails, the issue is RDS connectivity (Security Group or secret values).

Error 9: CloudWatch Additions Have Broken References

The Errors

1. **Undefined `log` function** — Script will fail with `log: command not found`
2. **Log path mismatch** — CloudWatch Agent won't collect app logs

Problem 1: Undefined `log` Function

Where it's broken:

Your CloudWatch sections use `log "..."` but that function is never defined in your script:

```
log "Installing CloudWatch Agent..."
log "Creating CloudWatch Agent configuration..."
log "Starting CloudWatch Agent..."
log "Enabling CloudWatch Agent to start on boot..."
```

What happens: The script will crash with `log: command not found` when it hits those lines.

How to fix it — Step by Step:

1. Open your `user_data.sh` file
2. Find these 4 lines and replace them:

Find this line	Replace with this line
<code>log "Installing CloudWatch Agent..."</code>	<code>echo "\$(date '+%Y-%m-%d %H:%M:%S') - Installing CloudWatch Agent..."</code>
<code>log "Creating CloudWatch Agent configuration..."</code>	<code>echo "\$(date '+%Y-%m-%d %H:%M:%S') - Creating CloudWatch Agent configuration..."</code>
<code>log "Starting CloudWatch Agent..."</code>	<code>echo "\$(date '+%Y-%m-%d %H:%M:%S') - Starting CloudWatch Agent..."</code>
<code>log "Enabling CloudWatch Agent to start on boot..."</code>	<code>echo "\$(date '+%Y-%m-%d %H:%M:%S') - Enabling CloudWatch Agent to start on boot..."</code>

1. Save the file

Problem 2: Log Path Mismatch

Where it's broken:

- Your Flask app writes logs to: `/var/log/notes-app.log` (a **file**)
- Your CloudWatch config expects: `/var/log/notes-app/*.log` (a **directory** with files inside)

What happens: CloudWatch Agent will never find your app logs because it's looking in the wrong place.

How to fix it — Step by Step:

1. Open your `user_data.sh` file
2. Find this line near the top:

```
LOG_FILE="/var/log/notes-app.log"
```

1. Replace it with:

```
LOG_FILE="/var/log/notes-app/app.log"
```

1. Find the line that says `# Create application directory` and **add this line right after it:**

```
mkdir -p /var/log/notes-app
```

1. Find this section inside the Flask app Python code (inside the `FLASK_APP` heredoc):

```
logging.FileHandler('/var/log/notes-app.log'),
```

1. Replace it with:

```
logging.FileHandler('/var/log/notes-app/app.log'),
```

1. Save the file

Verification After Fixes

After applying both fixes, your `user_data.sh` should have:

- ✓ `LOG_FILE="/var/log/notes-app/app.log"` (not `/var/log/notes-app.log`)
- ✓ `mkdir -p /var/log/notes-app` before the app directory creation
- ✓ Python FileHandler pointing to `/var/log/notes-app/app.log`
- ✓ All 4 `log "..."` lines replaced with `echo "$(date ...)"` format



Once these fixes are in place:

1. Terminate your current EC2 instance
2. Run `terraform apply` again with the updated `user_data.sh`
3. The new instance will bootstrap correctly with working CloudWatch logging